



4

18

2700 sp

نظري

محتوى مجاني غير مخصص للبيع التجاري

كلية الهندسة المعلوماتية

السنة الرابعة

AND OR GRAPH

د. باسل الخطيب



RB Informatix; 26/10/2025

خوارزميات البحث الذكية

المقدمة

في المحاضرات السابقة تعاملنا مع مسائل كان حلها يعتمد على إيجاد مسار داخل شجرة او غراف بحث، أي اننا عند محاولة حل عقدة معينة كنا نكتفي بالوصول الى أحد أبنائها الذي يوفر لنا الحل المطلوب؛ وهذا كان كافيا في تلك الأنواع من المسائل. لكن هناك فئات أخرى من المسائل لا يمكن حل العقدة فيها باختيار ابن واحد فقط بل يجب حل مجموعة من الأجزاء معا ثم دمج نتائجها للوصول إلى الحل النهائي؛ كما يحدث في العديد من المسائل الرياضية التي تتطلب حساب عدة حدود قبل تركيبها للحصول على الإجابة الكاملة. هذا النوع من المسائل هو ما يقودنا الى استخدام بنية ال and-or graph والتي تساعدنا على تمثيل المشكلات التي تحتاج حولا جزئية مترابطة او بدائل متعددة.

الان سنتعرف الى أداة من اهم الأدوات المستخدمة في الذكاء الاصطناعي لحل المشكلات المركبة وهي ال and-or graph. حيث تقوم هذه الأداة بتمثيل أي مشكلة معقدة عن طريق تقسيمها الى أجزاء أصغر وفهم العلاقات بين الحلول الممكنة.

مثال 1:



في حال كان لدينا أكثر من طريقة للحل كيف يمكن ان نمثل المسألة؟
هنا لدينا مسألة بسيطة وهي "الحصول على جهاز تلفزيون"

ال main goal الحصول على ال TV

الطرق الممكنة للحل (OR):

سرقه ال TV

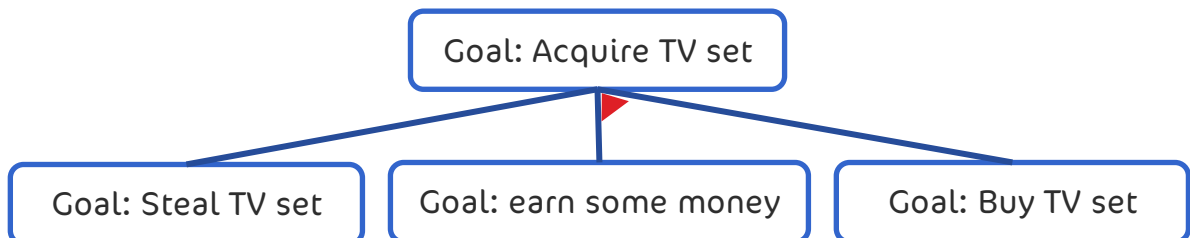
الحصول على مال وشراء TV

هناك خطوات يجب تنفيذها معا (AND) عند اختيار طريق معين

فمن أجل الحصول على التلفاز يجب علينا ان نحصل على النقود ثم نشترى التلفاز

الفكرة الأساسية هنا:

ال or يشير الى طرق بديلة للحل بينما ال and يشير الى خطوات يجب تنفيذها معا لحل المسألة.



الشكل العام لـ AND/OR Graph:

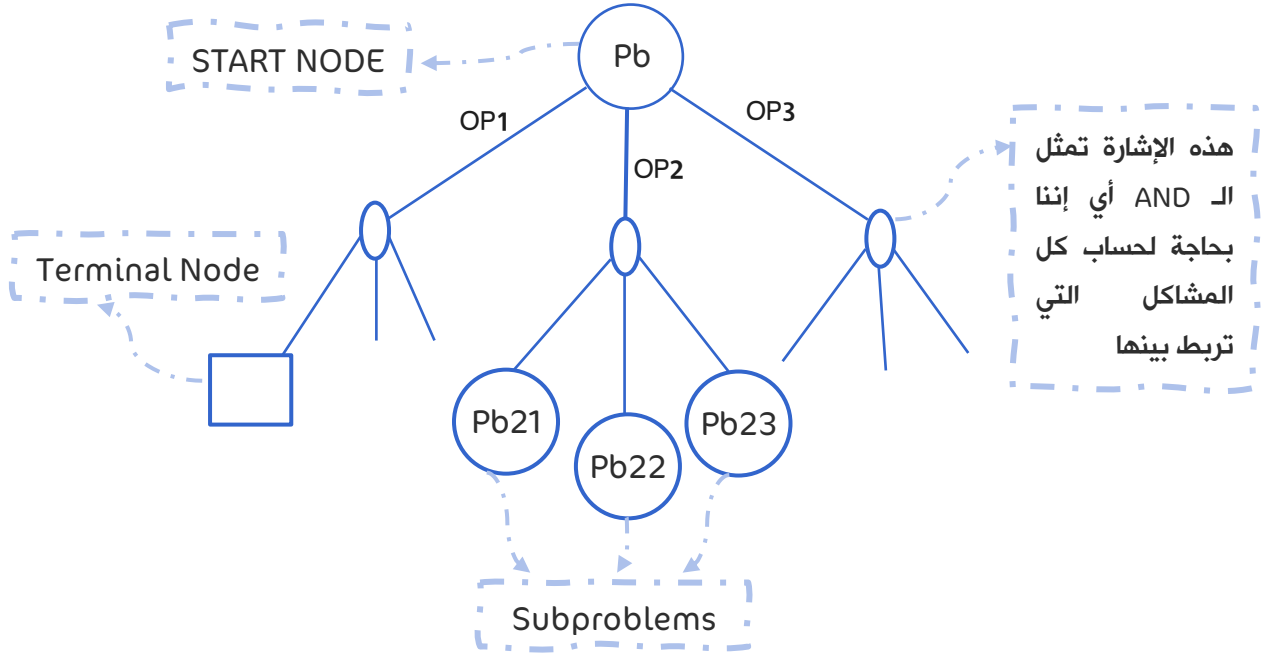
العقد تمثل المشكلات أو ال subproblems

روابط AND تعني أن المشكلة يتم حلها فقط عند حل كل العقد الأبناء نقصد هنا المشكلات الأبناء.

روابط or: تمثل حلول بديلة للمشكلة.

ال Start Node: المشكلة الاصلية وهي الحالة التي تبدأ منها المسألة

ال terminal node: هي مشاكل جزئية محلولة.

**لاحظ:** بين OP1, OP2, OP3 يوجد or هذا يعني أنه لحل Pb يوجد ثلاث حلول بديلة

مثال 2:

في هذا المثال سنرى كيف يمكن حل المعادلات الرياضية باستخدام ال AND OR GRAPH

ملاحظة: غالبا ما يكون هناك أكثر من طريقة لإعادة تمثيل المعادلة او اختزالها

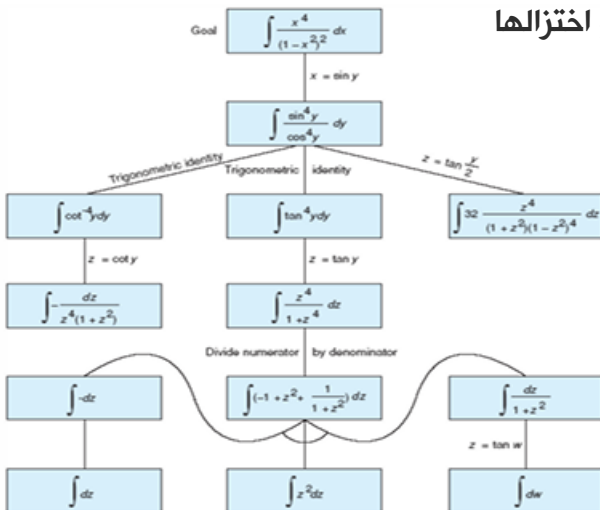
الآن لنرى مثال معادلة التكامل التالية

طبعا في هذا المثال ليس الهدف ان نصل لقيمة التكامل

وانما كيف بإمكاننا تحويل التكامل المعقد الى تكاملات

جزئية للوصول للحل عن طريق الـ

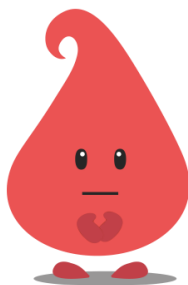
AND/OR Graph.



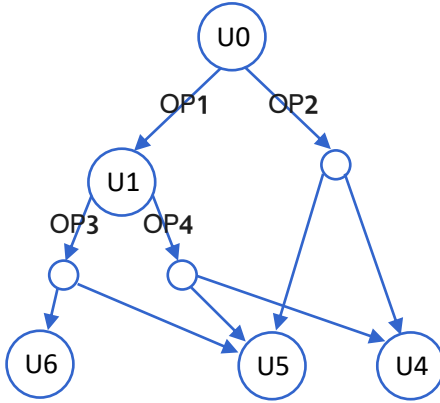
تعاريف هامة

- Node represent subproblems: العقد تمثل مشكلات فرعية
المعنى ان كل عقدة في الغراف تمثل حالة أي مشكلة جزئية من المشكلة الاصلية حيث ليس كل عقدة تعني حل نهائي بعضها يحتاج الى تفكيك لمشاكل فرعية أكثر.
 - And links represent subproblems decompositions: روابط AND تعني تقسيم المشكلة الى مشاكل جزئية يجب حلها معا جميعا
المعنى عندما نرى رابط AND من عقدة اب الى عدة أبناء فهذا يعني ان لحل العقدة الأب لابد من حل كل أبنائها أي النتائج الجزئية تجمع للحصول على الحل الكلي
 - OR link represent alternative solution: روابط OR تمثل حلولاً بديلة
المعنى أن رابط OR من عقدة أب إلى مجموعة أبناء يعني أن أي واحد من الأبناء يكفي لحل العقدة الأب أي نختار بديلاً واحداً لحل المشكلة
 - Start node is initial problem: نقطة البداية هي المشكلة الاصلية
هذه العقدة التي نبدأ البحث او التحليل منها
 - Terminal nodes are solved subproblems: العقد النهائية هي مشكلات محلولة
العقد التي ليس لديها أبناء تمثل مشاكل محلولة اما تكون بسيطة او قيم ثابتة او حقائق أساسية
مثلا في مشكلة حسابية قد تكون هذه العقد تحمل قيم عددية ثابتة (ليست حلول أي الوصول اليها لا يعني اننا قمنا بحل المشكلة الاصلية وانما هي مجرد قيم ثابتة) او نتيجة عملية حسابية تظهر كحل نهائي.
 - ماذا نعني بال Solution graph:
ليكون الغراف الذي لدينا هو مخطط حل يجب ان تتوفر فيه الشروط التالية:
1. الحل الذي نبحث عنه يجب ان يبدأ من المشكلة الاصلية ال start node
 2. كل الأوراق في المخطط يجب ان تكون مشكلات جزئية محلولة Terminal
 3. إذا اخترنا طريق يحوي رابط and فلا يكفي ادراج العقدة فقط بل يجب ادراج جميع أبنائها أي أنه لحل العقدة يجب حل جميع أبنائها معاً
 4. إذا اخذنا طريق يحوي رابط or فنأخذ ابن واحد من البدائل لحل العقدة لا داعي لإدراج كل الأبناء

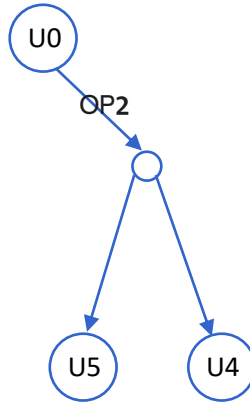
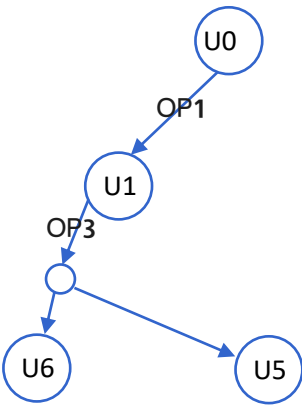
بالمختصر مخطط الحل Solution Graph
هو الشجرة الفرعية التي تحتوي على الحل الصحيح



■ مثال 3:



نلاحظ هنا انه لحل المشكلة الاصلية سيحتاج لدي مخططي حل فرعيين يبدأان من العقدة الاصلية لماذا مخططان؟ لأن لدينا رابط or أي لحل هذه المشكلة اما نحل الابن الأيسر او الابن الأيمن ونلاحظ انه لحل المسألة الاصلية إذا اخذنا الطريق اليميني يجب حل المشكلتين الفرعيتين U5 , U4 اما لو اخذنا الطريق اليساري فلحل المشكلة الأصلية يجب حل المشكلة الفرعية U1 و التي لحها يجب حل إما U4 U5 أو U6



■ مثال 4: Counterfeit Coin Problem

لدينا 8 عملات، واحدة منها مزيفة ويمكن أن تكون أثقل أو أخف، نملك ميزان كفتين، وكل وزنة تعطي 3 نتائج ممكنة:

- الكفة اليسرى أثقل
 - الكفة اليمنى أثقل
 - الميزان متوازن
- كل وزنة إذن تقسم الاحتمالات إلى 3 فروع.

■ المطلوب:

كم عدد الوزنات اللازمة لضمان اكتشاف العملة المزيفة؟

■ الشرح

■ أولاً: لماذا نحتاج 3 وزنات للوصول إلى الحل؟

لان كل وزنة تعطي 3 احتمالات وبالتالي 3 وزنات تعطي: $3^3 = 27$ نتيجة ممكنة

بينما عدد الاحتمالات الفعلية: $8 \times 2 = 16$ (أخف او أثقل)

إذاً 3 وزنات تغطي كل الاحتمالات بسهولة

■ ثانياً: أين ال AND، OR في GRAPHAO؟

OR: كل احتمال يخلق Subproblem جديدة، أي اختيار الوزن الذي نريد تطبيقه هو عقدة OR (اختيار نوع

الوزنة 1:1 أو 2:2 أو 3:3 أو 4:4

نتيجة الميزان من بين النتائج الثلاث هو عقدة OR أيضا (يسار أثقل - يمين أثقل - توازن) لأن الميزان سيعطي نتيجة واحدة فقط من بين الثلاث، وليس كلهم معاً.

AND: تظهر ال and في معالجة مجموعة الاحتمالات داخل عقدة واحدة، أي داخل كل نتيجة علينا تحليل جميع الاحتمالات داخل نفس الفرع، وفي كل Subproblem عليك تقسيم العملات الى مجموعات (and)

■ ثالثاً: أنواع المقارنات الممكنة لاكتشاف القطعة المزيفة:

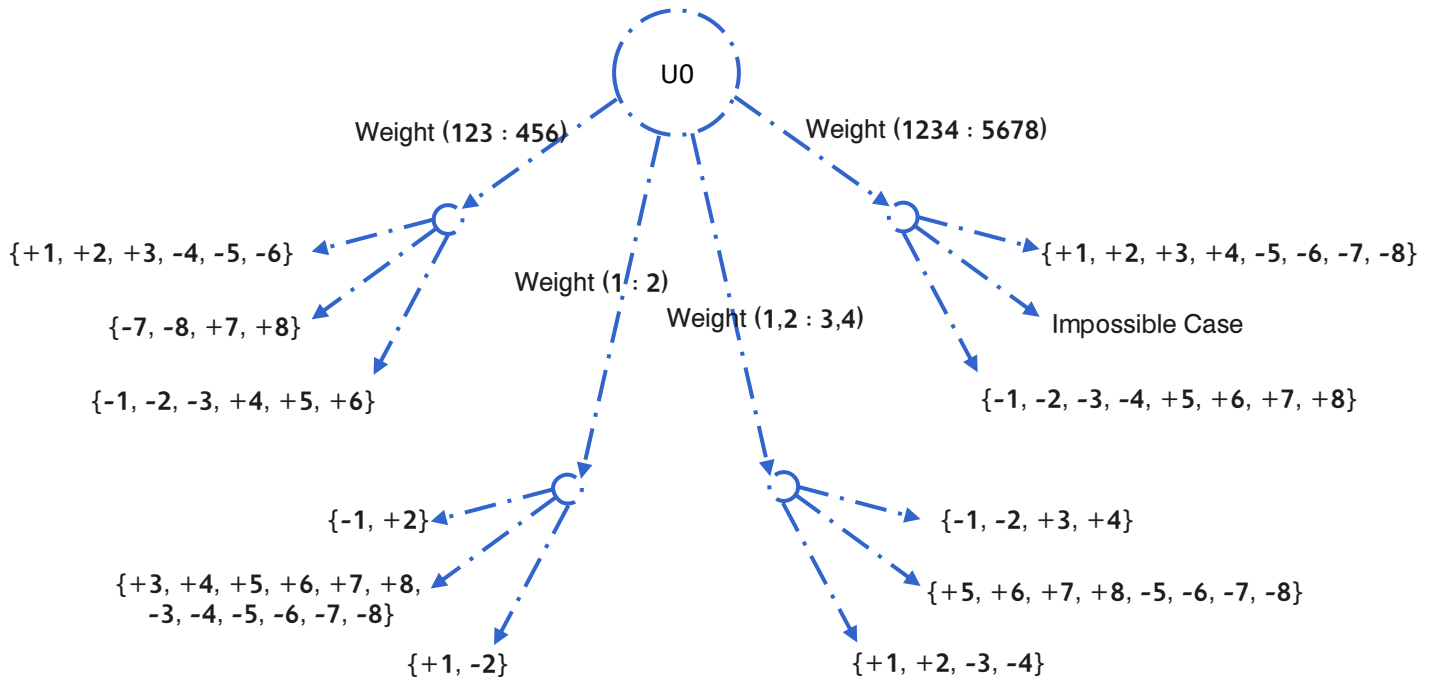
1:1 (وزن قطعة مع قطعة)

2:2 (وزن قطعتين مع قطعتين)

3:3 (وزن ثلاث قطع مقابل ثلاث قطع)

4:4 (وزن أربع قطع مقابل أربع قطع)

{+1, +2, +3, +4, +5, +6, +7, +8, -1, -2, -3, -4, -5, -6, -7, -8}



■ رابعاً: امثلة على أنواع الوزنات الممكنة:

لو وزنا 3 قطع مقابل 3 قطع مثال: 456 VS 123:

■ إذا توازن الميزان فهذا يعني أن العملات من 1 الى 6 سليمة والعملية المزيفة ستكون ضمن {7, 8}

كل منها اما - او + فيبقى أربع احتمالات {-8, +8, -7, +7}

■ إذا مال الميزان لجهة العملات 123 فهذا يعني أن أحد هذه العملات سيكون أثقل "+" او أحد الثلاث

عملات 456 سيكون أخف "-" أي سيكون لدي ست احتمالات {-6, -5, -4, +3, +2, +1}

■ وبالمثل لو مال لجهة ال 456, فهذه الوزنة ممتازة لأنها تقسم ال 16 احتمال الفعلي الى مجموعات صغيرة

■ لو وزنا 4 مقابل 4 مثال 5674 vs 1234

هذه الوزنة غير مفيدة لأن الميزان لن يتوازن أبداً (المزيفة حتما داخل هذه ال 8) فهكذا أصبح لدينا نتيجتان

فقط بدل 3 وهذا يجعل الغراف غير متناسق ويضيع علينا احتمال مهم من الاحتمالات كان يمكن ان يوصلنا للحل بأقل عدد من الخطوات لو تحقق، ولهذا نسمي التوازن هنا impossible case

والآن لنجيب على المسألة السابقة

ثلاث وزنات كافية لإيجاد القطعة المزورة (وبأقل عدد وزنات ضمني)؛ لأنه رياضيا كل وزنة تعطي ثلاث نتائج ممكنة (يسار أثقل / يمين أثقل / توازن)، وبعد K وزنات يمكن تمييز على الأكثر 3^K حالة مختلفة، وهنا عدد الحالات التي نحتاج تمييزها $2 \times 8 = 16$ حالة، أي نريد $3^K \geq 16$ ، ونلاحظ ان اقل K يحقق المعادلة هو 3. اذن وباختصار 3 وزنات هي الحد الأدنى الذي يضمن اكتشاف العملة المزيفة وتحديد ما إذا كانت اقل او أخف. نظريا لضمان اكتشاف العملة المزيفة بين 8 عملات باستخدام ميزان ذو كفتين نحتاج أن نخطط الوزنات بحيث كل وزنة تقلل عدد الاحتمالات التي بقيت أمامنا، ولأن كل وزنة تعطي ثلاث نتائج ممكنة (الكفة اليسرى أثقل – الكفة اليمنى أثقل – توازن)، فهي تسمح لنا في كل خطوة بتقسيم الاحتمالات إلى 3 مجموعات. وباستراتيجية وزن مدروسة (مثل 3 مقابل 3 في البداية)، يمكننا دائما تضيق الاحتمالات بحيث نصل للعملة المزيفة خلال 3 وزنات مهما كانت ظروف المسألة.

خوارزميات اختيار أفضل وزنة (Heuristics) في مسألة العملة المزيفة.

هذه الخوارزميات تجاوبنا على سؤال كيف نعرف الوزنة الأفضل لتقليل عدد الاحتمالات بأكبر قدر ممكن؟

أولاً: خوارزمية ال Minimize

القاعدة:

$$W(p) = \max(|uL|, |uE|, |uR|)$$

شرح الرموز:

- UL = عدد الاحتمالات التي تبقى اذا مالت الكفة اليسرى
- UR = عدد الاحتمالات التي تبقى اذا مالت الكفة اليمنى
- UE = عدد الاحتمالات اذا توازن الميزان

نعلم أن كل وزنة تعطي ثلاث نتائج وفي كل نتيجة يظهر لدينا عدد معين من الاحتمالات (مجموعة).

الفكرة من الخوارزمية: نريد ان نختار الوزنة التي تصغر أكبر مجموعة من هذه المجموعات الثلاث، أي لا نريد وزنة ينتج عنها وزنة اسوا فيها عدد كبير من الاحتمالات

لذلك باستخدام القانون نأخذ الوزنة الأفضل وهي التي تملك أصغر قيمة لهذا الماكس



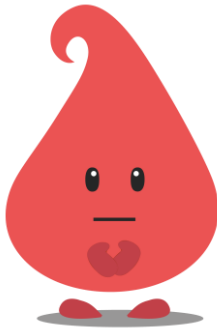
ثانياً: خوارزمية ال Information Entropy:

القاعدة:

$$I(p) = \log|uL|/|u| + \log|uR|/|u| + \log|uE|/|u|$$

هذه طريقة أخرى لقياس جودة الوزن، مستخدمة في نظرية المعلومات (Information Theory) حيث تقوم على فكرة:

الوزن المثالية هي التي توزع الاحتمالات بشكل متساوي بين نتائج الميزان الثلاث فكلما كان التوزيع متوازناً، يكون الـ entropy أعلى وبالتالي الوزن أفضل، ولكن ما هو الـ ENTROPY؟



كل مقارنة (1:1 أو 2:2 أو 3:3 أو 4:4) لها قيمة Entropy:

- $I(\text{Weight}(1,2)) = 1.298$
- $I(\text{Weight}(12,34)) = 1.375$
- $I(\text{Weight}(123,456)) = 1.861$ ← الأفضل
- $I(\text{Weight}(1234,5678)) = 0.75$ ← الأسوأ

لماذا 3:3 أفضل بحسب ENTROPY؟

لأنه يوزع الاحتمالات على نتائج الميزان الثلاث بشكل قريب من التوازن، جزء منها يذهب للاحتمال L، جزء لـ R، وجزء للتوازن، مما يعني أن كل احتمال يصبح له مسار منطقي دون تضخم في فرع معين. بينما 4:4 تعطي نتيجتان فقط (ميل اليسار أو ميل اليمين)، ونتيجة التوازن مستحيلة، لذلك الـ ENTROPY منخفض جداً ← وزنة سيئة

مثال 5: Producing Rules

في هذا المثال سنركز على نقطتين أساسيتين:

1. أي مسألة يمكن التعبير عنها باستخدام مجموعة قواعد

2. يمكن تمثيل هذه القواعد باستخدام الـ and or graph

الآن لدينا مجموعة القواعد التالية ونريد معرفة الحلول الممكنة لحل المسألة d

هنا كل قاعدة تمثل:

▪ عقدة and

مثلاً لحل d يجب أن نحل (a, e, f) معا

▪ عقدة بديلة or

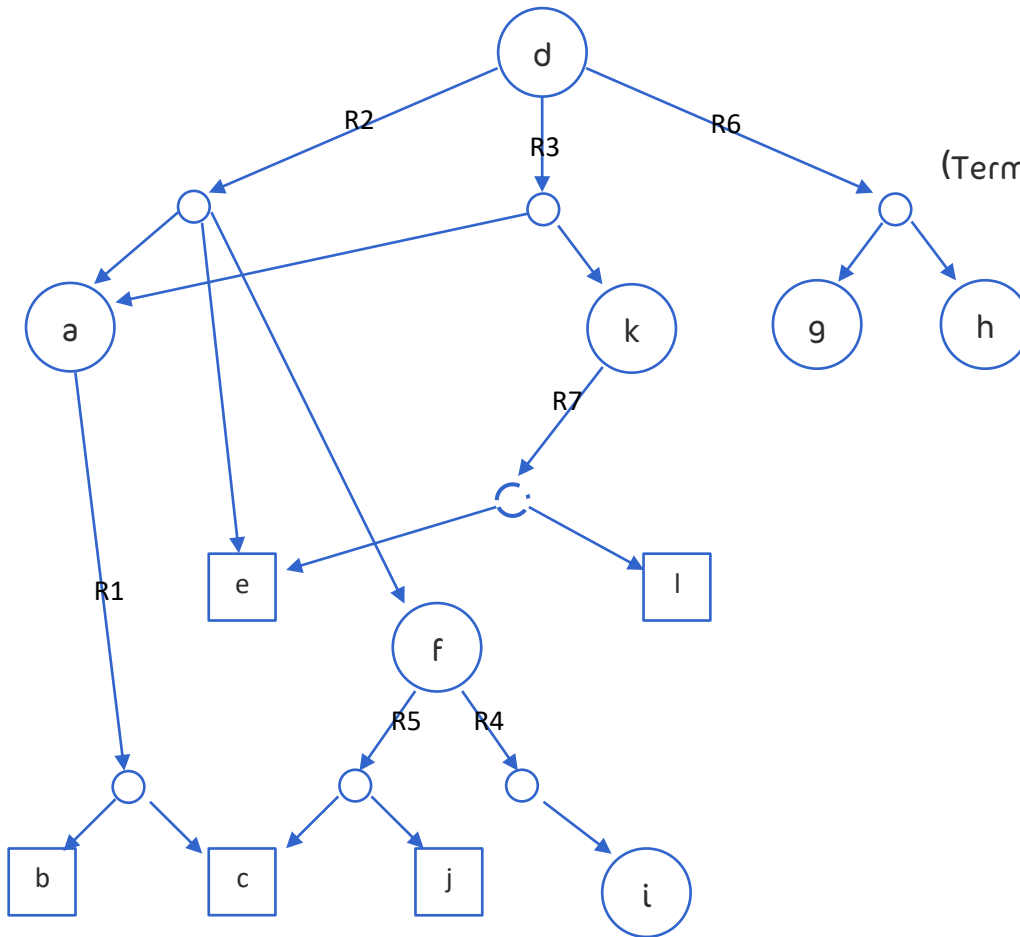
لأن لحل d يوجد أكثر من قاعدة بديلة يمكن تطبيقها

هذا يعني لحل المسألة d سنستخدم إما القاعدة R3 أو R2 أو R6 لكن لو قررنا اختيار القاعدة R2 فيجب حل المسائل الجزئية a، e، f

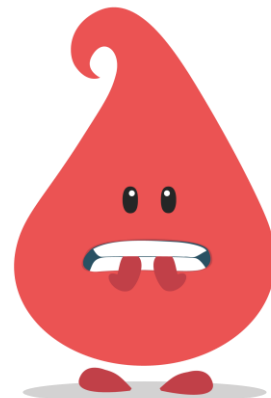
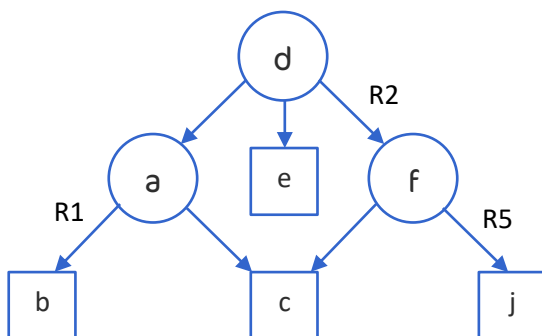
فإذا الحل هو استخراج تسلسل القواعد المراد تطبيقها

ملاحظة:

المستطيلات النهائية هي مسائل
جزئية بسيطة محلولة (Terminal nodes)

**الحل**

1. لو أخذنا R2 فهل هي الحل؟؟
نعم لأن 'a' لها حل عن طريق حل 'b, c' بأخذ القاعدة R1 و 'e' نهائية، و 'f' لها حل عن طريق حل 'c, j' بأي بأخذ القاعدة R5
انتبه: هنا لم نأخذ القاعدة R4 لأنها لا تصل الى مسألة نهائية محلولة
هنا تسلسل القواعد يكون R1 -> R5 -> R2
ال SOLUTION GRAPH الأول يكون كالتالي:

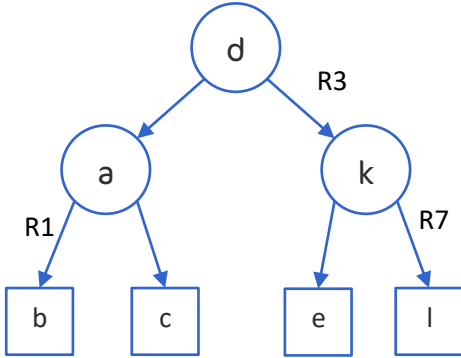


2. لو اخذنا R3 هل هي الحل؟؟

نعم لان a لها حل عن طريق حل c, b بأخذ القاعدة R1, و k لها حل بأخذ القاعدة R7

هنا تسلسل القواعد يكون R1 -> R7 -> R3

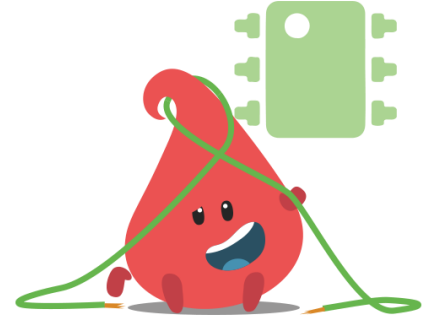
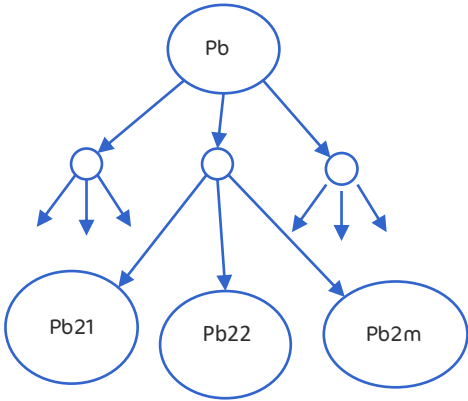
ال Solution Graph الثاني يكون كالتالي:



ملاحظة:

الخوارزمية الكلاسيكية لحل مسائل AND/OR هي ال Backtracking فهي تبحث في كل الاحتمالات: تجرب حلا وإذا فشل ترجع للخلف وتجرب حلا بديلا حتى تجد كل الحلول الممكنة. فكمما تعلمنا في ال PROLOG يعتمد على خوارزمية ال Backtracking حيث تبحث عن حلول الاستعلام: تختار قاعدة، تحاول إثبات الشروط الفرعية (AND)، وإذا فشلت تتراجع لتجربة قاعدة بديلة (OR) حتى تجد كل الحلول الممكنة

الشكل العام ل AND-OR GRAPH



قاعدة تمثيل هيكل الحل كمجموعة من الاحتمالات:

- Pb هي المسألة الأصلية
- كل فرع من or يمثل بديلا ممكنا لحل المسألة
- داخل كل or هناك and يمثل مجموعة شروط فرعية يجب تحقيقها معا حتى يعتبر هذا البديل صحيح

Solve (Pb) => OR (

(AND (Solve (SPb11),...,Solve(SPb1n)))

(AND (Solve (SPb21),...,Solve(SPb2m)))

(AND (Solve (SPb31),...,Solve(SPb3k)))

)

General Rule

القاعدة العامة لحل المسألة:

GR: (P, OP, AND[GR1, ..., GRm])

- P هي المسألة الحالية مثل (Pb)
- op هو نوع العملية (and/or) التي تربط الحلول الفرعية
- AND [GR1, ..., GRm] هي قائمة القواعد الفرعية أو الحلول الجزئية.

مثال اريسييزي:

- بفرض المسألة Pb هي "السفر من A الى C"
- البدائل الممكنة:
- رحلة مباشرة من A الى C
- الرحلة عبر (A → B → C)
- الرحلة عبر (A → D → C)
- تمثيل هيكل الحل كمجموعة من الاحتمالات:

Solve (Pb) => OR (
AND (Solve(A->C)),
AND(Solve(A->B), Solve(B->C)),
(AND (Solve(A->D), Solve(D->C))
)

تمثيله باستخدام GR

GR: (A->C, OR, [
(A->C, AND, []),
(B, AND, [
(A->B, AND, []),
(B->C, AND, [])
]),
(D, AND, [
(A->D, AND, []),
(D->C, AND, [])
])
])

```

SEARCH-AND/OR(P)
  If Final(P) Then Search-AND/OR = nil
  If DeadLock (P) Or Depth(P) < Limit Then
    Search-AND/OR = FAIL
  LOperators = Ordered_Applicable_Operators(P)
  ReturnPoint:
    If LOperators = nil Then
      Search-AND/OR = FAIL
    OP = Head (LOperators)
    {Sp1,Sp2,...,Spn} = SubProblems(OP,P)
    LOperators = Queue (LOperators)
    Escape=False; i=1
    While i<=n And Not Escape do
      Begin
        Depth(Spi)=Depth(Sp)+1
        i=i+1
        GRi=Search-AND/OR(Spi)
        If GRi=FAIL Then Escape=True
      End
    If Escape Then GoTo ReturnPoint
  Serach-AND/OR=(P,OP,AND(GR1,...,GRn))

```



شرح أسطر الكود:

- 1 Search-AND/OR(P)
الدالة تأخذ المشكلة P لتبدأ البحث عن حل لها
- 2 If Final(P) then
3 Search-AND/OR = nil
فحص النهاية: إذا كانت المشكلة P محلولة بالفعل (وصلنا للهدف) اعد قيمة nil وهي تعني نجاح أي لا يوجد خطوات إضافية مطلوبة
- 4 If DeadLock(P) OR Depth(P) < Limit then
5 Search-AND/OR = FAIL
فحص الفشل: إذا وصلنا لطريق مسدود (Deadlock) أو تجاوزنا عمق البحث المسموح به (Limit)، فهذا المسار فاشل. نرجع FAIL.
- 6 LOperators = Ordered_Applicable_Operators(P)

جلب الخيارات: قم بجلب قائمة بكل العمليات أو الخطوات الممكنة لحل هذه المشكلة، ورتبها في القائمة LOperators.

7 ReturnPoint:

نقطة الرجوع: هذه مجرد علامة (Label) لنعود إليها لاحقاً إذا فشلت المحاولة الحالية وأردنا تجربة خيار آخر.

8 If LOperators = NIL then

9 Search-AND/OR = FAIL

نفذ الحل: لو القائمة فارغة (أو جربنا كل الخيارات وفشلت)، إذن لا يوجد حل للمشكلة. نرجع FAIL.

10 OP = Head(LOperators)

اختيار عملية: خذ أول عملية (Operator) من القائمة لنجربها.

11 {Sp1, Sp2, ..., Spn} = SubProblems(OP, P)

التقسيم: عند تطبيق العملية OP على المشكلة P، فإنها تتفكك إلى مجموعة مشاكل صغيرة فرعية (Sub-problems).

12 LOperators = Queue(LOperators)

تحديث القائمة: احذف العملية التي اخترناها للتو من القائمة (بحيث إذا فشلت ننتقل إلى الخيار التالي في المحاولة القادمة).

13 Escape = False

14 i = 1

تجهيز العداد: نجهز متغير اسمه Escape (للخروج في حال الفشل) ونبدأ العداد i من 1 للمرور على المشاكل الفرعية.

15 While i ≤ n AND NOT Escape do

حلقة التكرار (AND Loop): سنمر على المشاكل الفرعية الواحدة تلو الأخرى، طالما أننا لم ننتهِ وطالما لم يحدث فشل (Not Escape).

هنا الجزء الخاص بـ AND: لازم كل المشاكل الفرعية تنجح.

16 Depth(Spi) = Depth(Sp) + 1

زيادة العمق: نحدد عمق المشكلة الفرعية الحالية بزيادة 1 عن المشكلة الأصلية (لأننا نزلنا مستوى في الشجرة).

17 i = i + 1

نزد العداد 1.

18 GRi = Search-AND/OR(Spi)

الاستدعاء الذاتي (Recursion): هذا أهم سطر. الدالة تنادي نفسها مرة أخرى لتقوم بحل المشكلة الفرعية Spi.

19 If GRi = FAIL then

20 Escape = True

شرط الفشل: لو نتيجة حل المشكلة الفرعية كانت FAIL، نقوم بتفعيل وضع الهروب Escape=True.
(لأن في منطق AND، لو جزء واحد فشل، الخطة كلها تفشل).

21 End

نهاية حلقة التكرار (While).

22 If Escape then

23 GoTo ReturnPoint

المحاولة مرة أخرى (OR Logic): لو خرجنا من الحلقة و Escape مفعّل (يعني فشلنا في حل الأجزاء الفرعية)، نعود فوراً لـ ReturnPoint (السطر 5) لنجرب عملية أخرى من القائمة LOperators.

24 Search-AND/OR=(P, OP, AND(GR1, ..., GRn))

النجاح: لو وصلنا هنا، فهذا يعني أن Escape ما زال False (كل المشاكل الفرعية تم حلها بنجاح).
نُرجع الحل النهائي، وهو يتكون من: المشكلة + العملية التي نجحت + حلول جميع المشاكل الفرعية.

AO star - AO*

هي خوارزمية Informed search تهدف الى إيجاد أفضل حل في AO Graph تشبه في عملها الى حد كبير خوارزمية A* حيث انها تعتمد على حساب كلفة العقدة من خلال جمع الكلف المتراكمة لها حتى الان مع الـ Heuristic، وكما في الـ A* فان الـ Heuristic من الأفضل ان يكون underestimate.

رغم ذلك هناك بعض الاختلاف في طريقة العمل لان هذه الخوارزمية تهدف الى إيجاد subgraph وليس path

آلية عمل الخوارزمية

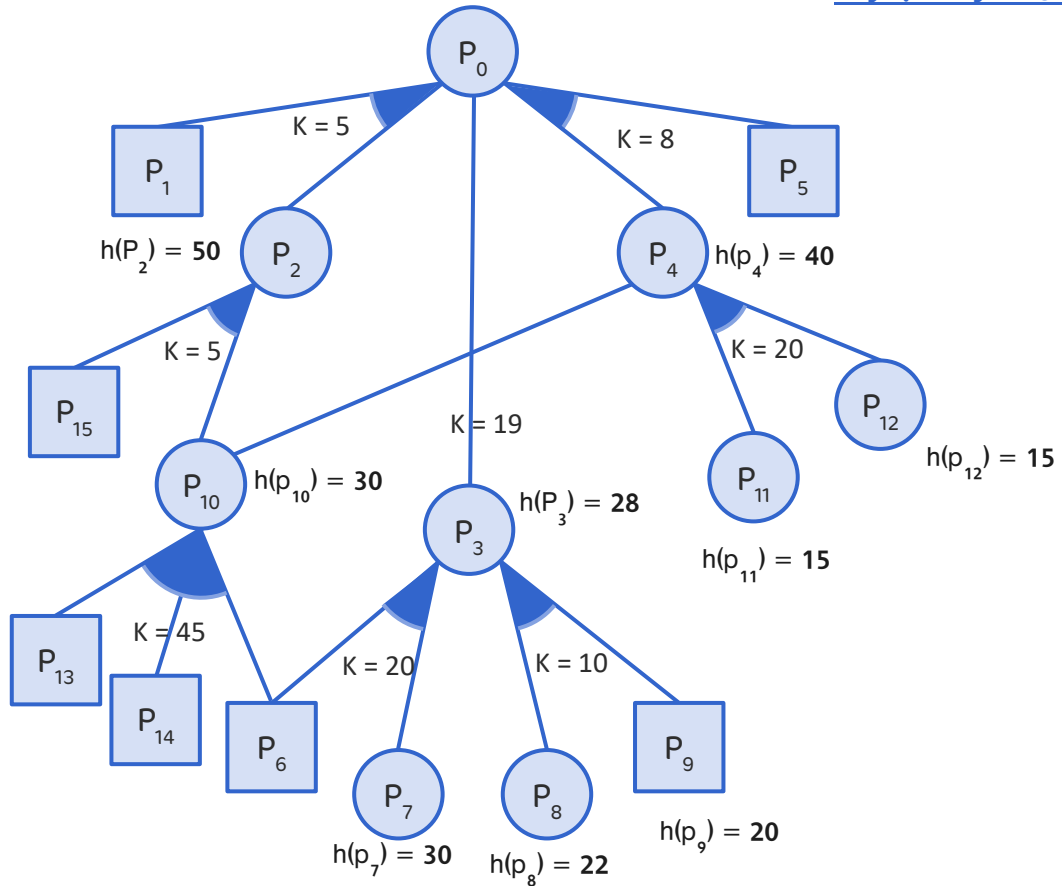
1. تبدأ الخوارزمية من العقدة الأساسية root
2. تقوم بفحص قيم الأبناء واختيار الابن صاحب الكلفة الأقل
3. تقوم بفحص قيمة العقدة والتأكد من قيمتها بناء على قيمة heuristic العقد الأبناء وكلفة الانتقال إليهم في حال كانت قيمة العقدة فعلاً مساوية لقيمة الـ heuristic المسجل لها نتابع العملية لاختيار الابن الأصغر، في حال كانت قيمة العقدة أكبر من الـ heuristic نحدث قيمة العقدة ونقوم بتحديث جميع القيم بدءاً منها ووصولاً الى root ومن ثم نعيد تكرار الخوارزمية بدءاً من الخطوة الأولى.
4. نكرر الخوارزمية حتى نصل الى مرحلة جميع العقد التي نفحص قيمها هي أوراق Terminal أي ليس لديها heuristic فنوقف ونكون قد وجدنا الحل

مثال 1:

- المسألة الأساسية المراد حلها هي الـ p0 نمثلها بجذر الشجرة
- الهدف هو إيجاد الـ Sub Solution Graph للمسألة الأساسية

القيمة التجريبية للمسائل الجزئية	تكلفة تطبيق كل قاعدة	قواعد المسألة (العمليات)
$H(P_2) = 50$	$K(OP_1) = 5$	$OP_1: P_0 \rightarrow P_1 \& P_2$
$H(P_3) = 28$	$K(OP_2) = 19$	$OP_2: P_0 \rightarrow P_3$
$H(P_4) = 40$	$K(OP_3) = 8$	$OP_3: P_0 \rightarrow P_4 \& P_5$
$H(P_7) = 30$	$K(OP_4) = 5$	$OP_4: P_2 \rightarrow P_{15} \& P_{10}$
$H(P_8) = 22$	$K(OP_5) = 20$	$OP_5: P_3 \rightarrow P_6 \& P_7$
$H(P_9) = 20$	$K(OP_6) = 10$	$OP_6: P_3 \rightarrow P_8 \& P_9$
$H(P_{10}) = 30$	$K(OP_7) = 10$	$OP_7: P_4 \rightarrow P_{10}$
$H(P_{11}) = 15$	$K(OP_8) = 20$	$OP_8: P_4 \rightarrow P_{11} \& P_{12}$
$H(P_{12}) = 15$	$K(OP_9) = 45$	$OP_9: P_{10} \rightarrow P_{13} \& P_{14} \& P_6$

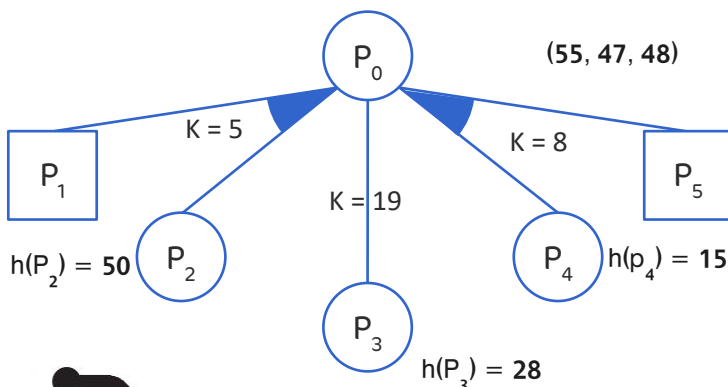
الآن لنمثل القواعد بغراف

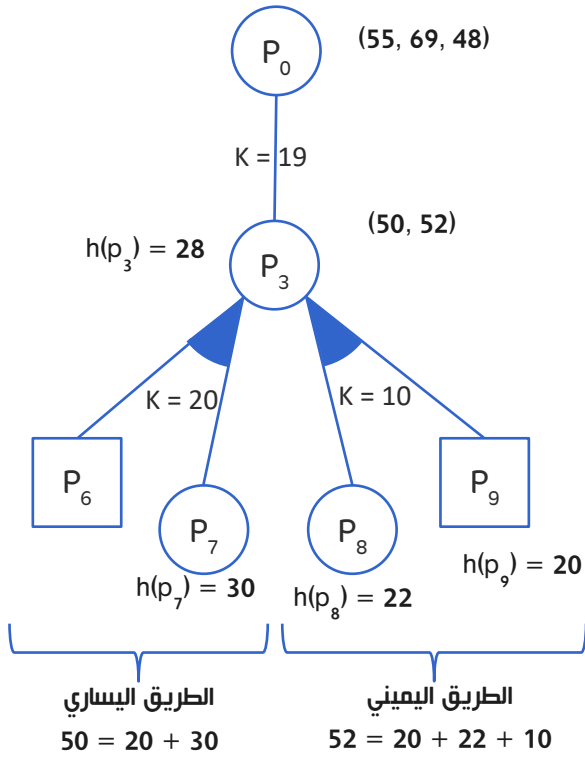


الآن نبدأ البحث عن الحل

1. نبدأ من الجذر:

نقوم بحساب تكلفة كل طريق حيث تكلفة كل طريق تساوي تكلفة العملية المطبقة + القيمة التجريبية heuristic
فنتج لدينا على الترتيب {55,47,48}





2. نختار الان الطريق ذو التكلفة الأقل وهو المؤدي ل

P3

3. نحسب قيمة P3 بناءً على قيمة heuristic العقد

الأبناء وكلفة الانتقال إليهم في الطريق اليميني أولاً

فنجد انها

$$52 = 22 + 20 + 10 = h(p_8) + h(p_9) + 10$$

نحسب قيمة P3 بناءً على قيمة heuristic العقد

الأبناء وكلفة الانتقال إليهم في الطريق اليساري

فنجد انها

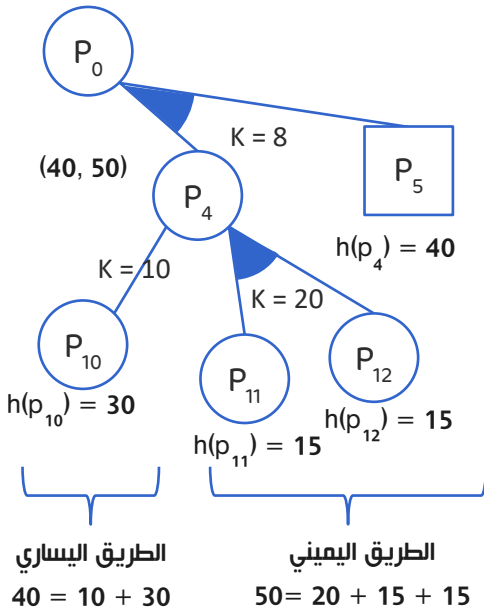
$$50 = 0 + 30 + 20 = h(p_6) + h(p_7) + 20$$

نأخذ القيمة الأصغر من الطريقين وهي 50 ونقارنها

مع $h(p_3)$ فيظهر لدينا انها أكبر

4. فنحدث قيم العقد وصولاً للجذر

$$69 = 19 + 50$$



5. نعود الى الجذر ونختار الطريق ذو التكلفة الأقل نجد انه

الطريق المؤدي الى P4

6. نفحص قيمة P4 فنجد انها:

في الطريق اليميني $20 = h(p_{12}) + h(p_{11}) + 20$

$$50 = 15 + 15$$

في الطريق اليساري $40 = 30 + 10 = h(p_{10}) + 10$

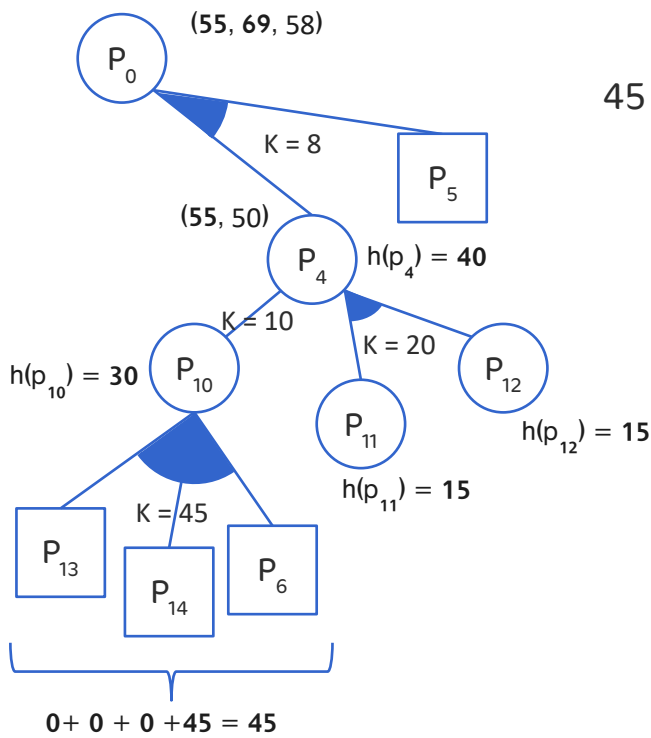
نأخذ القيمة الأصغر من الطريقين وهي 40 ونقارنها مع

$h(p_4)$ فيظهر انها تساويها

فتتابع الى الـ P10 ونختارها ونفتحها

Celebrate the
efforts not
just the wins.





7. نفحص الان قيمة ال p_{10} وهي

$$45 + 0 + 0 + 0 = h(p_{13}) + h(p_{14}) + h(p_6) + 45$$

نقارنها مع ال $h(p_{10})$ لنجد انها أكبر

8. فنحدث قيم العقد وصولا للجذر

أولا تكلفة الطريق من P_4 الى P_{10} تصبح:

$$55 = 45 + 10$$

وتكلفة الطريق من P_0 الى P_4 هي

$$58 = 50 + 8$$

اخذنا ال 50 لأنها الأقل كلفة، لان تكلفة الطريق

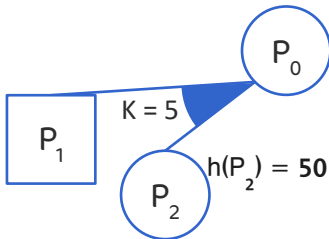
من P_4 الى p_{10} هي

$$55 = 45 + 10$$

وتكلفة الطريق من P_4 الى العقدة P_{11} و P_{12} هي

$$50 = 15 + 20 + 15$$

9. الان نعود للجذر ونأخذ الطريق ذو التكلفة الأقل وهو الطريق المؤدي الى p_1, p_2



10. نفحص العقدة P_2 فقط لان ال P_1 هي Terminal

نجد ان قيمة P_2 هي

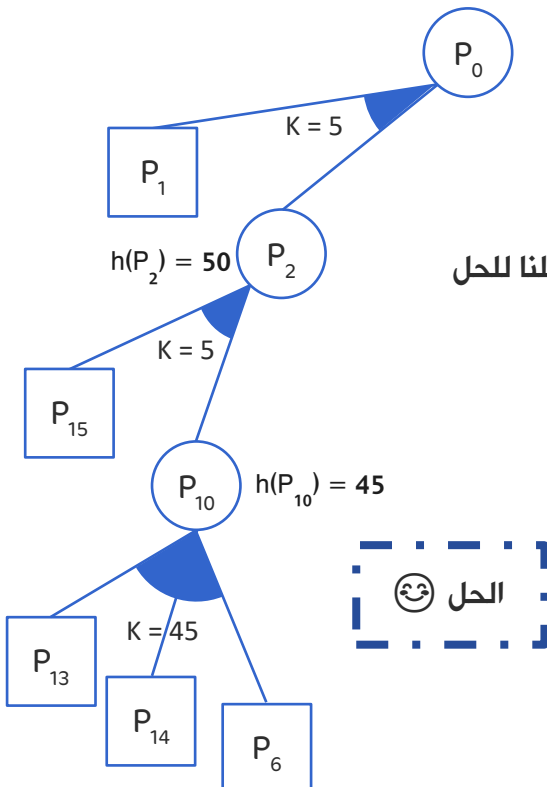
$$50 = 0 + 45 + 5 = h(p_{15}) + h(p_{10}) + 5$$

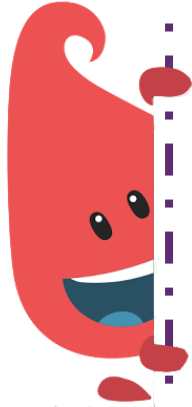
نقارنها مع $h(p_2)$ نلاحظ انها تساويها

11. نجد اننا وصلنا الى حالة كل الأوراق هم Terminal وبالتالي وصلنا للحل

تكلفة الطريق كاملة هي 55

فيكون الحل هو ال Subgraph الموضح بالشكل





```

OPEN = {InitialProblem}
PENDING = {InitialProblem}
CLOSE = nil
Repeat
  GR-Solution = ChooseGRSolution_from_OPEN.
  PENDING = Leaves (GR-Solution)
  P = ChooseProblem(PENDING)
  Delete(P,PENDING), Add(P,CLOSE)
  For each OPi in ApplicableOperators(P)
    For each SPij in SubProblems(OP,P)
      Add(SPij, OPEN)
  fi(P)= k(OPi) +  $\sum h(Spij)$ 
  ModifyAncestor(P)
Until PENDING=Nil Or AllPrimary(PENDING)
IF PENDING=Nil Then FAIL

```

شرح أسطر الكود

- 1 OPEN = {InitialProblem}
القائمة المفتوحة: نضع فيها المشكلة الأساسية (الجذر) لنبدأ منها. هذه القائمة ستحتوي على كل العقد التي تم اكتشافها وتشكل "الرسم البياني" الحالي.
- 2 PENDING = {InitialProblem}
قائمة الانتظار: هذه قائمة بـ "العقد التي لم توسع بعد". نضع فيها المشكلة الأساسية لأننا لم نحلها بعد.
- 3 CLOSE = nil
قائمة المغلقة: نهز قائمة للعقد التي انتهينا من توسيعها بالكامل، وهي فارغة حالياً.
- 4 Repeat
بداية الحلقة: سنكرر الخطوات التالية حتى نصل للحل أو نفشل.
- 5 GR-Solution = ChooseGRSolution_from_OPEN.
تحديد المسار الواعد: من بين كل المسارات الموجودة في OPEN، حدد "الرسم البياني الجزئي" (Partial Solution) الذي يمتلك أقل تكلفة تقديرية حالياً. يعني: "ركز لي على أفضل خطة عندنا لحد الآن".
- 6 PENDING = Leaves (GR-Solution)
تحديد الأطراف: انظر إلى نهاية هذا المسار الواعد، وحدد العقد الطرفية (Leaves) التي لم نقوم بتوسيعها بعد. هذه هي العقد التي تحتاج لعمل.

- 7 `P = ChooseProblem(PENDING)`
اختيار عقدة: اختر واحدة من العقد الطرفية لنعمل عليها، ولنسمها P.
- 8 `Delete(P,PENDING), Add(P,CLOSE)`
نقل العقدة: بما أننا سنعمل على P الآن، احذفها من قائمة الانتظار PENDING وأضفها لقائمة CLOSE (لأنها ستصبح عقدة موسعة)
- 9 `For each OPi in ApplicableOperators(P)`
تجربة العمليات: سنمر على كل عملية أو خيار متاح (O_{Pi}) يمكن تطبيقه على المشكلة P.
- 10 `For each SPij in SubProblems(OP,P)`
المشاكل الفرعية: كل عملية قد تنتج عدة مشاكل فرعية (S_{Pij}) (لأنها عقدة AND). سنمر عليهم واحدة واحدة.
- 11 `Add(SPij, OPEN)`
إضافة للرسم: أضف هذه المشاكل الفرعية الجديدة للرسم البياني OPEN (عشان نعرف إنها موجودة).
- 12 $f_i(P) = k(O_{Pi}) + \sum h(S_{Pij})$
حساب التكلفة (أهم سطر): نحسب تكلفة الحل عبر العملية الحالية O_{Pi}:
k(O_{Pi}): التكلفة الفعلية للقيام بالخطوة.
 $\sum h(S_{Pij})$: مجموع التكلفة التقديرية (Heuristic) لكل المشاكل الفرعية الناتجة. (نجمعهم لأنهم AND، لازم نحلهم كلهم).
- 13 `ModifyAncestor(P)`
تحديث الأسلاف (Back-propagation): بما أننا حسبنا تكلفة جديدة لـ P، يجب أن نخبر "أباها" وجدها بتغير التكلفة.
إذا أصبحت P مكلفة جداً، قد يقرر الأب تغيير المسار واختيار حل آخر (تغيير المؤشر للأفضل).
- 14 `Until PENDING=Nil Or AllPrimary(PENDING)`
شرط التوقف: استمر في التكرار حتى يحدث واحد من اثنين:
PENDING=Nil: لا يوجد عقد متبقية للعمل عليها.
AllPrimary(PENDING): (هنا المعنى الضمني) أن أفضل مسار (Primary Solution) أصبحت كل عقده محلولة (SOLVED) ولا يوجد شيء في الانتظار يخص المسار الأفضل.
- 15 `IF PENDING=Nil Then FAIL`
النهاية: لو خرجنا من الحلقة لأن الـ PENDING فارغة ولم نصل لحل، إذن المشكلة ليس لها حل (فشل).

YOU ONLY FAIL
WHEN YOU STOP TRYING

